

Chapter 2. Git & Agile



Lee Youngkon



Basic concept

- ✓ What is Git?
 - Distributed Version Management System (DVCS)
 - Managing Source Code Change History
 - Can be developed by multiple people at the same time
- ✓ GitHub?
 - Git Storage Hosting Services
 - Collaborative platform
 - Code Management + Issue Management + CI/CD

Need for Git

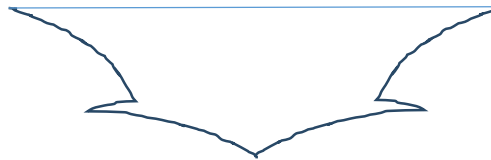
✓ Previously, copy files and perform version management

✓ Problems:

- Don't know which files are up to date
- I don't know who modified it
- Difficulty recovering previous versions
- Difficult to work with multiple people at the same time

Example

report.doc
report_final.doc
report_final_real.doc
report_final_real2.doc
report_final_real2_last.doc



git automatically manages all changes history

Git Features

- ✓ Each developer has a complete repository: Distributed Version Control System (DVCS)
- ✓ Manage Change History
- ✓ Track who, when, and what you modified
- ✓ Branch Features
- ✓ Provide independent development space: development without affecting each other
- ✓ Quick recovery: can be recovered even if accidentally deleted
- ✓ Collaboration Support: Multiple people can develop at the same time

procedure for using git

1 Initialize Repository



```
git init
```

Create a new Git repository in your project directory.

2 Create / Modify Files



Example: `Hello.java`

Create new files or edit existing files.

3 Stage Changes



```
git add .
```

Add changes to the staging area.

4 Commit Changes



```
git commit -m  
"Initial commit"
```

Save the changes to the local repository.

5 Connect to Remote Repository



```
git remote add origin  
https://github.com/user/  
project.git
```

Connect your local repository to a remote repository.

6 Push to Remote



```
git push origin main
```

Upload your local commits to the remote repository.

7 Collaborate



Team members clone, make changes, and push their work.

8 Pull / Update



```
git pull origin main
```

Download the latest changes from the remote repository.

9 Resolve Conflicts (if any)



If conflicts occur, merge and resolve the issues.

10 Repeat the Cycle



Continue the cycle of edit, commit, push, and collaborate.

git core concept

✓ Repository

- Space where Git stores version information
- Git Repository if you have a .git folder
- Working Directory (Work Directory)
- Space that the developer is currently working on
- Not yet saved to Git

```
Project
├── src
├── pom.xml
└── .git
```

Hello.java Modify

✓ Staging Area

- Spaces to save temporarily before committing
- `git add .`

```
Working Directory
    ↓
Staging Area
```

git core concept

✓ Commit

- Snapshots at a certain point in time
- `git commit -m "Login completed"`

✓ Branch

- independent development space
- Can develop multiple functions simultaneously

```
graph TD; main --- login; main --- payment; main --- delivery;
```

main
├── login
├── payment
└── delivery

git core concept

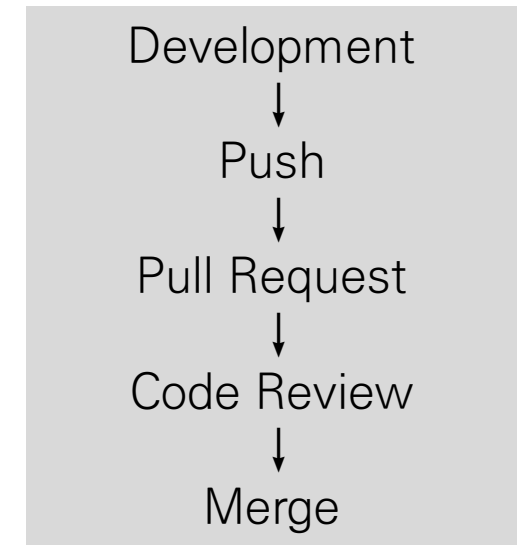
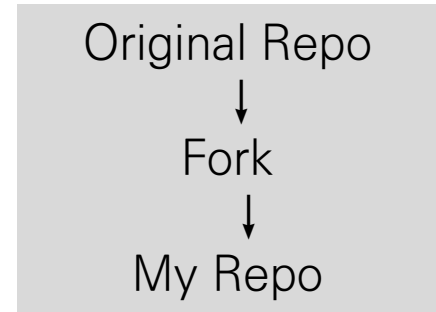
- ✓ HEAD
 - Pointer pointing to the branch or commit you are currently working on
- ✓ Checkout / Switch
 - command to change branch
 - `git switch login` $\Rightarrow$ main $\rightarrow$ login
- ✓ Merge
 - Working to combine branches
 - `git merge login` $\Rightarrow$ main branch login branch merge
- ✓ Conflict
 - Occurs when you modify the same file at the same time (when Merge)

git core concept

- ✓ Remote Repository
 - Remote Storage
 - example: GitHub, GitLab, Bitbucket
- ✓ Clone
 - Copying remote storage
 - `git clone https://github.com/user/project.git`
- ✓ Push
 - Upload My Commit to the Server
 - `git push origin main`

git core concept

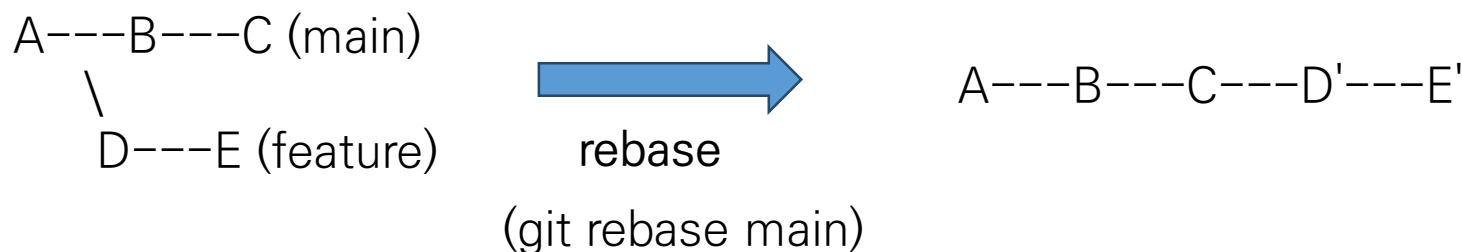
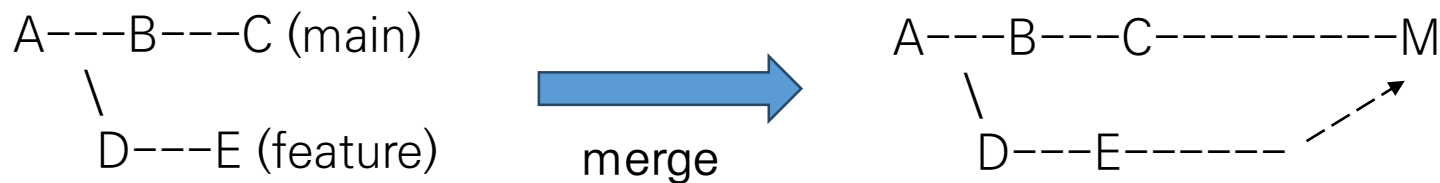
- ✓ Pull
 - Get the latest information on the server
 - `git pull origin main`
- ✓ Fork
 - Copy someone else's repository to my account
- ✓ Pull Request (PR)
 - The process of reviewing and requesting the code to merge



git core concept

✓ Rebase

- Putting my work back on top of the latest version
- The history is clean
- Existing commit hash changed; risk in collaborative branches



git core concept

- ✓ Cherry-pick
 - Import only specific commit:
 - Import bug fix commit only

main
A-B-C

git cherry-pick E



A-B-C-E

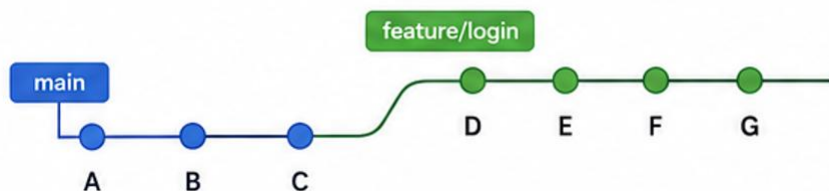
feature
A-B-D-E-F

git core concept

✓ Squash

- Combine multiple commitments into one
- Often used in practice when merging PR

Before Squash (Many Commits)



Commit History

A Initial commit
B Add user model
C Setup login page

D Add login form
E Validate input
F Connect to API
G Show error message

Squash



Combine
D, E, F, G
into 1 commit

After Squash (Single Commit)



Commit History

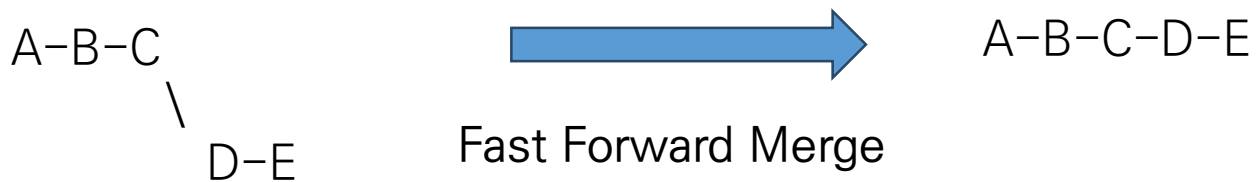
A Initial commit
B Add user model
C Setup login page

D' Login feature complete
(squashed commit)

- Add login form
- Validate input
- Connect to API
- Show error message

git core concept

- ✓ Fast Forward Merge
 - Branch is not branched
 - Git does it automatically



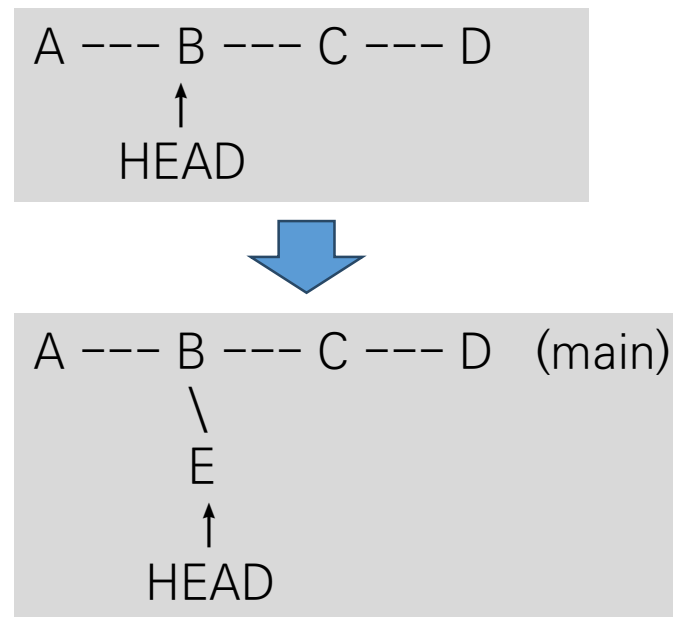
git core concept

✓ Detached HEAD

- HEAD indicates a specific commit, not a branch
- `git checkout 4f5e3d`
- HEAD => Commit
- Can be lost after creating a new commit

✓ Stash

- Temporary archive of what you're working on
- ① `git stash` ② `git stash pop`



git core concept

✓ Tag

- Show specific versions

- `git tag v1.0`

✓ Reset

- Reversing

- Example: A-B-C-D

- Remove D

- `git reset --hard HEAD~1`

- Result: A-B-C

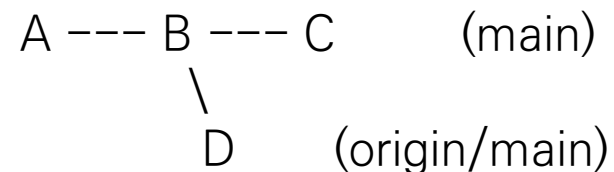
git core concept

✓ Revert

- Safe cancellation
- For example, cancel D from A-B-C-D
- `git revert D`
- Result: A-B-C-D-E, E cancels D (E cancels D commit)

✓ Fetch

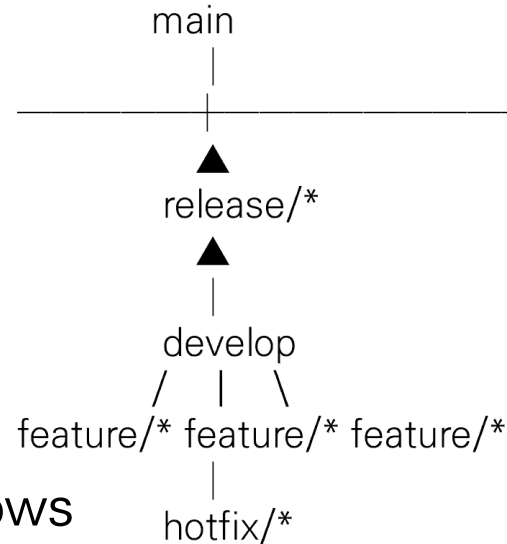
- Import server status info only
- `git fetch`



git core concept

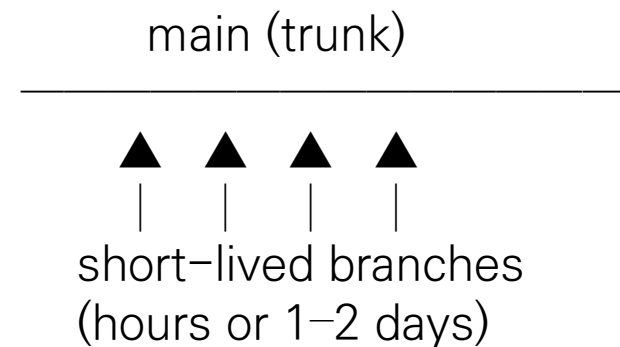
✓ Git Flow

- A branching strategy for software development
- Uses multiple long-lived branches
- Supports parallel feature development
- Separates development, release, and hotfix workflows



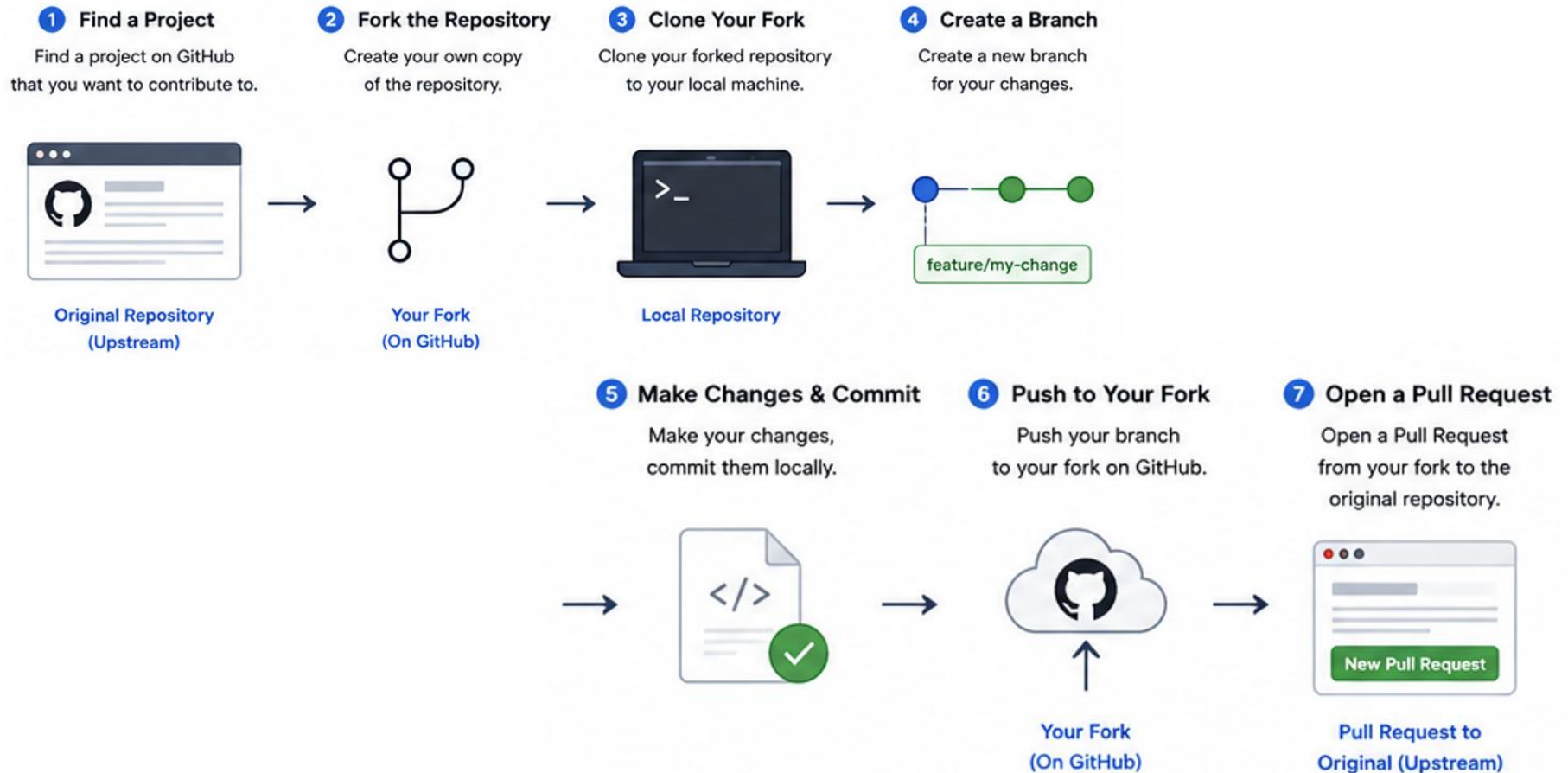
✓ Trunk-Based Development

- Single main (trunk) branch
- Short-lived feature branches
- Frequent and quick merges
- Small, incremental commits
- Continuous Integration (CI) friendly



git core concept

✓ Fork Workflow



git core concept

✓ Pull Request (PR) Review

1 Create a Branch

Create a new branch from the `main` branch.

2 Make Changes

Edit, add and commit your changes.

3 Push Changes

Push your branch to the remote repository.

4 Open a Pull Request

Go to the repository and open a Pull Request to merge your changes into `main`.

5 Code Review

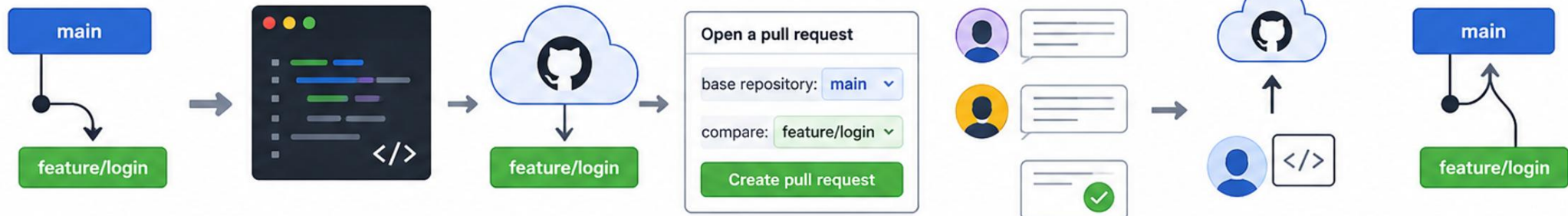
Team members review your code, comment and request changes if needed.

6 Address Comments

Update your code based on the review comments and push again.

7 Merge

Once approved, your changes are merged into `main`.



git core command

- ✓ Create and set up repositories

command	Description
git init	Create a new storage
git clone URL	Remote Storage Replication
git config --global user.name "name"	User Name Settings
git config --global user.email "email"	Email Settings

git core command

✓ Check Status

command	Description
git status	Check current status
git log	Check commit history
git log --oneline	Check a brief history
git diff	Check changes

git core command

- ✓ Add and commit files

command	Description
git add File name	Add File
git add .	Add a full file
git commit -m "Message"	Create Commit
git commit -am "Message"	Modified files automatically add + commit

git core command

✓ Remote Storage Connections

command	Description
git remote add origin URL	Register Remote Storage
git remote -v	Remote Storage Verification
git remote remove origin	Delete remote storage

✓ Push / Pull

command	Description
git push origin main	Uploading a Server
git pull origin main	Get the latest code
git fetch	Import remote status only

git core command

✓ Branch Management

command	Description
git branch	Branch List
git branch Branch name	Create Branch
git switch Branch name	Move Branch
git switch -c Branch name	Move after creation
git branch -d Branch name	Delete Branch

git core command

✓ Branch Management

command	Description
git branch	Branch List
git branch Branch name	Create Branch
git switch Branch name	Move Branch
git switch -c Branch name	Move after creation
git branch -d Branch name	Delete Branch

✓ Branch merge

command	Description
git merge Branch name	Merge Branch

git core command

✓ Stash

command	Description
git stash	Save current job temporarily
git stash list	Check the list
git stash pop	restoration

git core command

✓ Rebase

command	Description
git rebase main	Move the current branch up to the main latest status
git rebase --continue	Continue after conflict resolution
git rebase --abort	Cancellation

✓ Cherry-pick

git cherry-pick Commit ID

git core command

✓ (Tag)

command	Description
git tag	Tag list
git tag v1.0	Create a tag
git push origin v1.0	Uploading tags

✓ Revert

git revert Commit ID

git core command

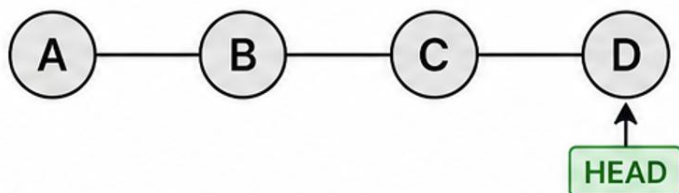
✓ Reset

command	Description
<code>git reset --soft HEAD~1</code>	When you write a commit message incorrectly
<code>git reset --mixed HEAD~1</code>	Maintain Repository, remove from staging area, maintain changes in working directory
<code>git reset --hard HEAD~1</code>	Maintain Repository, remove from staging area, remove from working directory

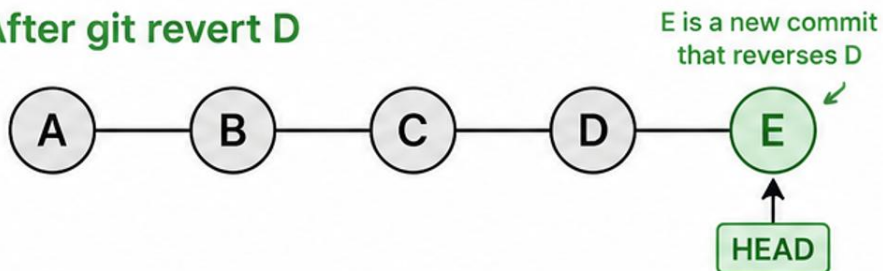
git revert

Create a new commit that undoes changes

Before



After git revert D

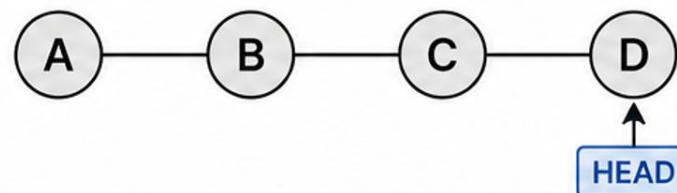


- ✓ Creates a new commit
- ✓ History is preserved
- ✓ Safe for shared repositories

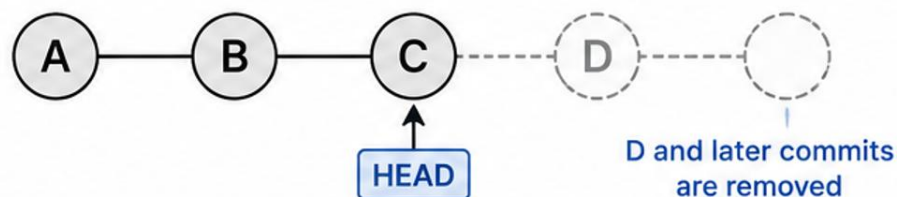
git reset

Move the branch pointer to a previous commit

Before



After git reset --hard C

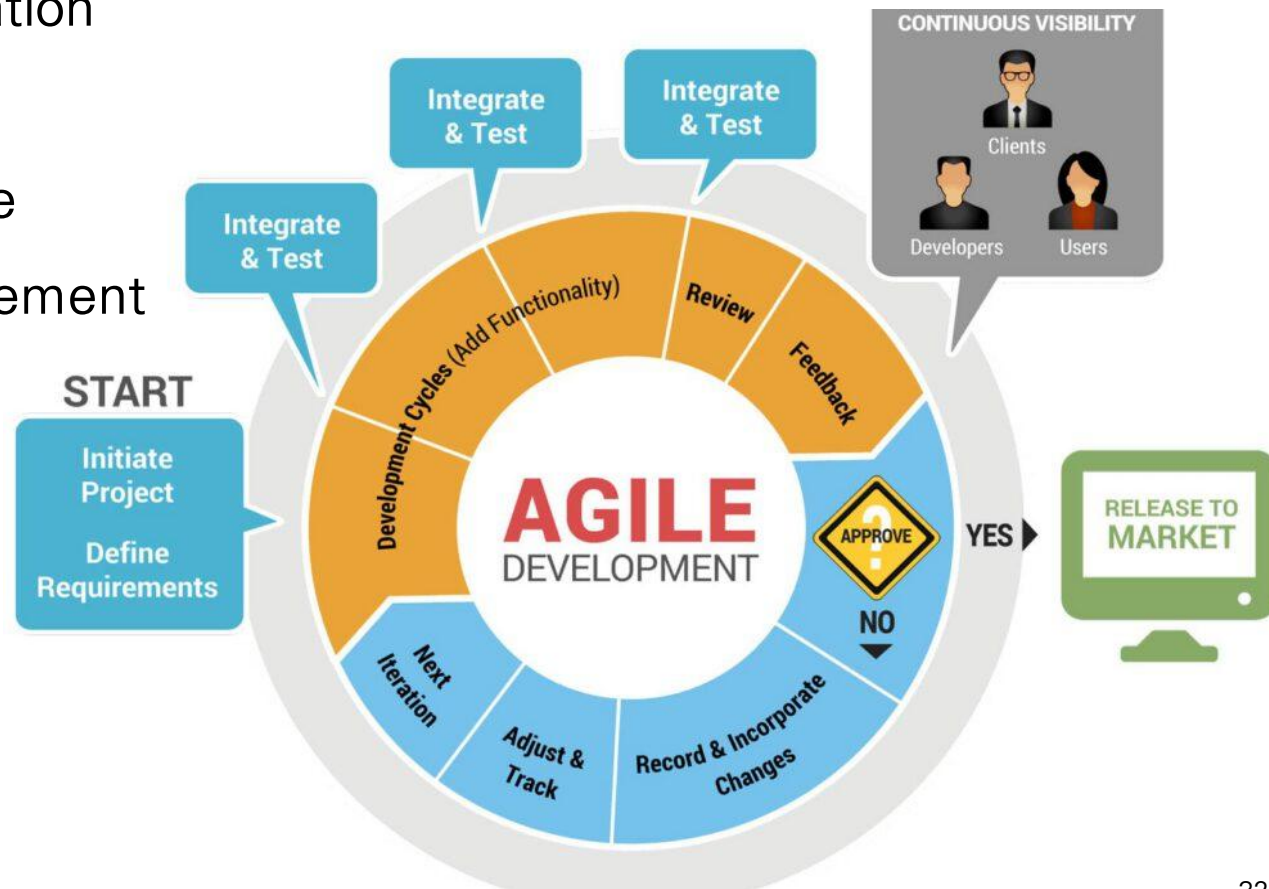


- ✗ No new commit
- ✗ History is rewritten
- ✗ Not safe after pushing (use with caution)

Definition

✓ Agile is a software development approach that focuses on:

- Customer collaboration
- Rapid delivery
- Flexibility to change
- Continuous improvement

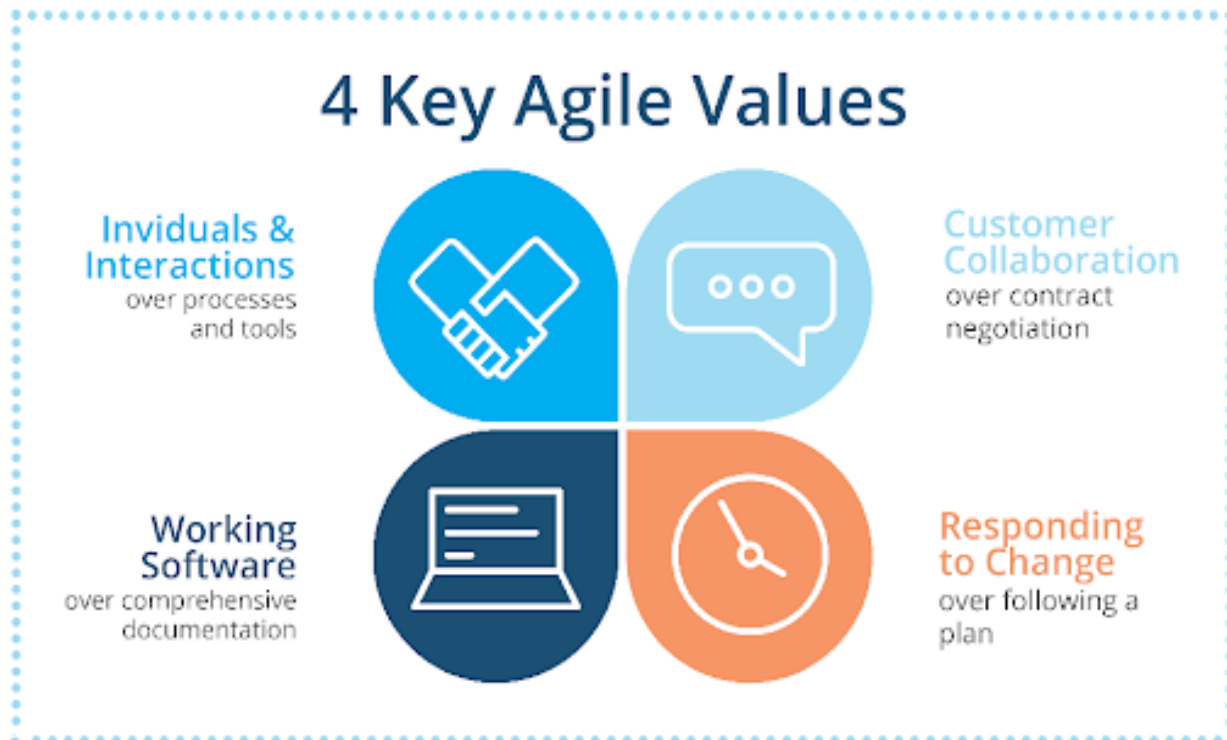


Traditional vs Agile

Traditional	Agile
Plan-driven	Adaptive
Fixed requirements	Evolving requirements
Long release cycle	Frequent releases
Documentation-focused	Customer-focused

Four Core Values

- ✓ Individuals and interactions over processes and tools
- ✓ Working software over comprehensive documentation
- ✓ Customer collaboration over contract negotiation
- ✓ Responding to change over following a plan



Deliver value to customers quickly and continuously.

12 agile principles in software development



Customer
satisfactions



Changing
requirements



Frequent
delivery



Communicate
regularly



Support
team member



Face-to-face
communication



Measure
work progress



Development
process



Good
design



Measure
progress

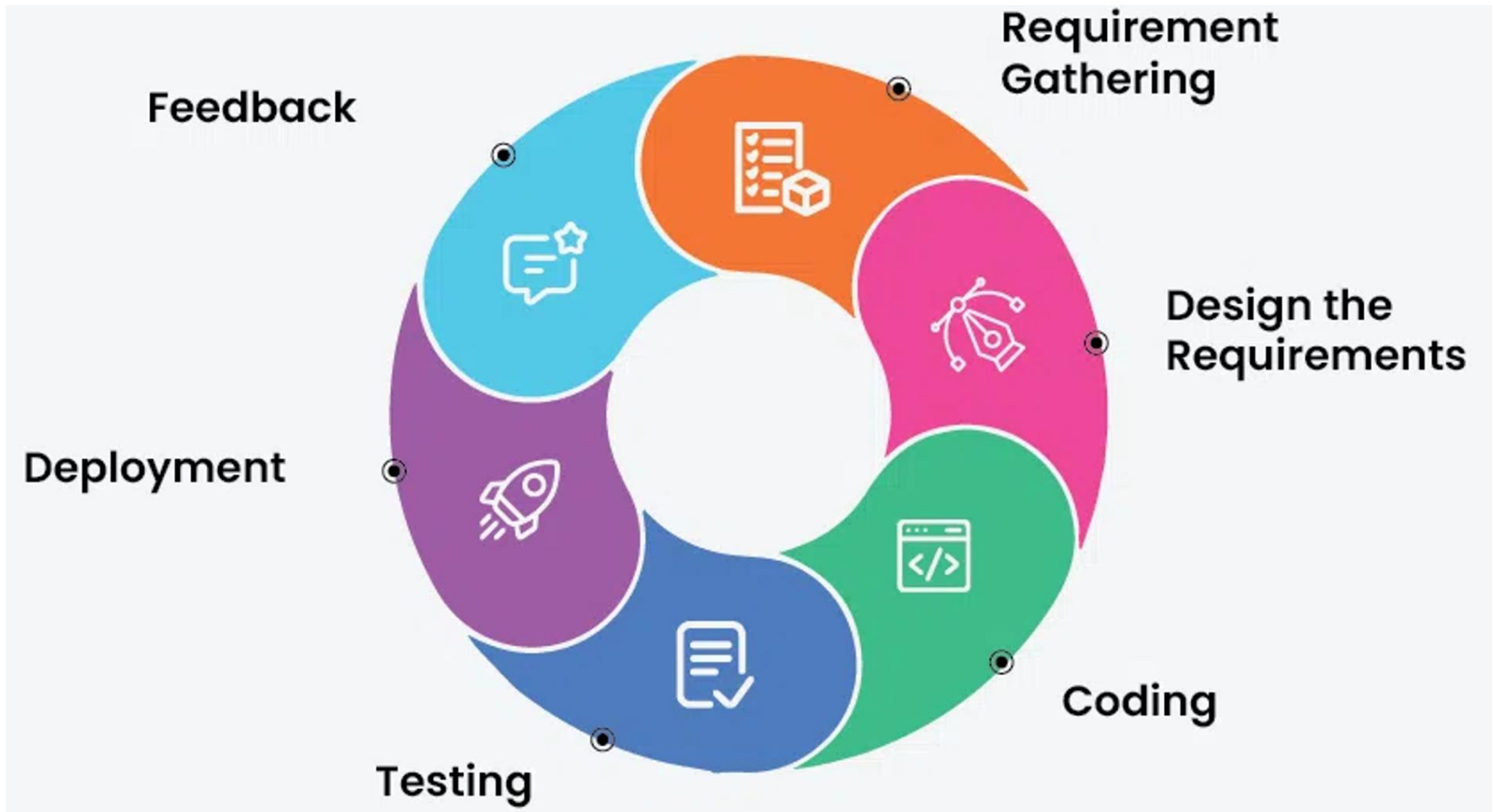


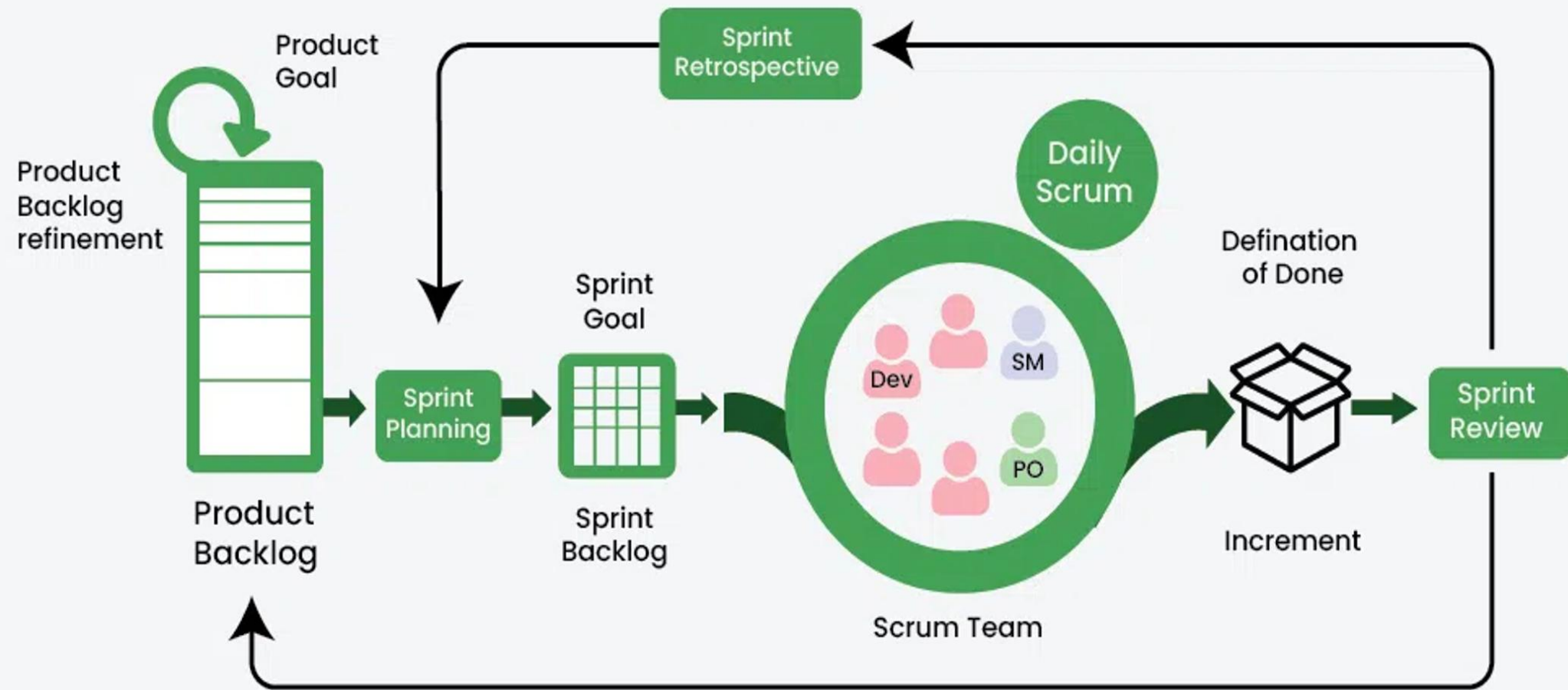
Continue
seeking result



Reflect and
adjust regularly

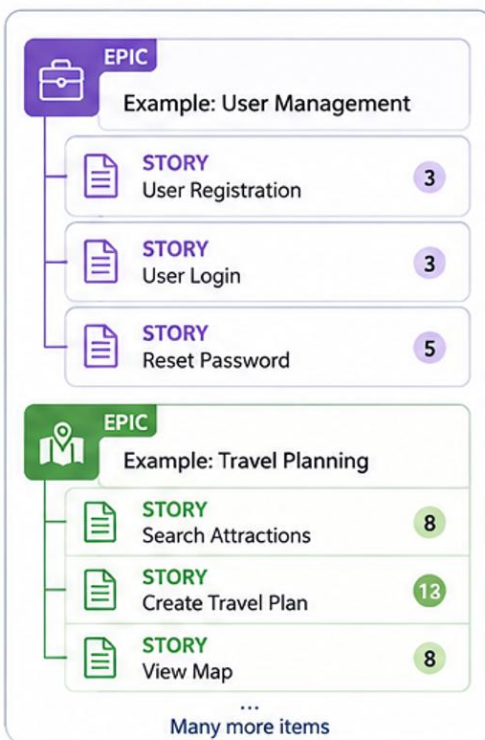
Agile Development Cycle





1. PRODUCT BACKLOG (WHAT to build)

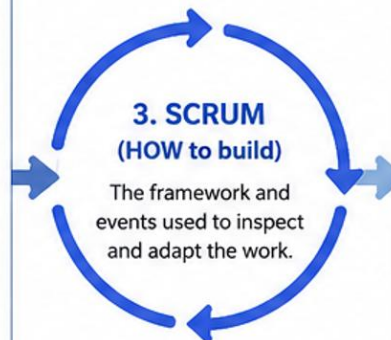
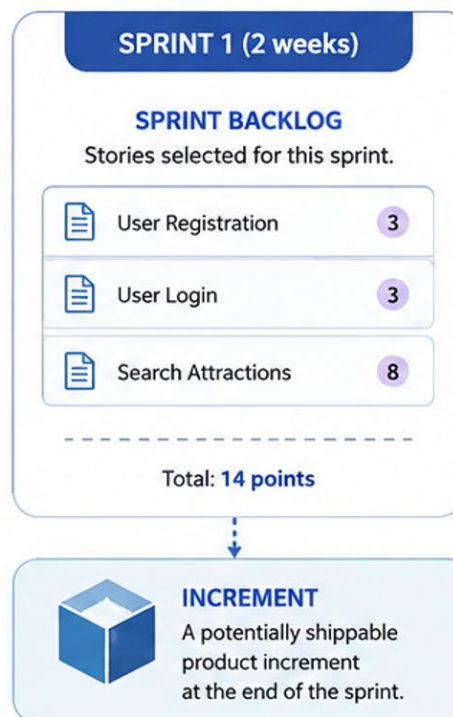
A prioritized list of everything that might be needed in the product.



Select
highest
priority
items

2. SPRINT (WHEN to build)

A time-boxed iteration (usually 1 ~ 4 weeks).



4. SCRUM EVENTS (Inspect & Adapt)

Regular events to ensure transparency, inspection, and continuous improvement.



KEY CONCEPTS



EPIC

Large body of work that can be broken down into smaller pieces.



STORY (User Story)

A short description of a feature from the user's perspective.



BACKLOG

The product backlog is the full list; the sprint backlog is the selected subset for the sprint.



SPRINT

A fixed time period to complete a set of stories and deliver increment.



SCRUM

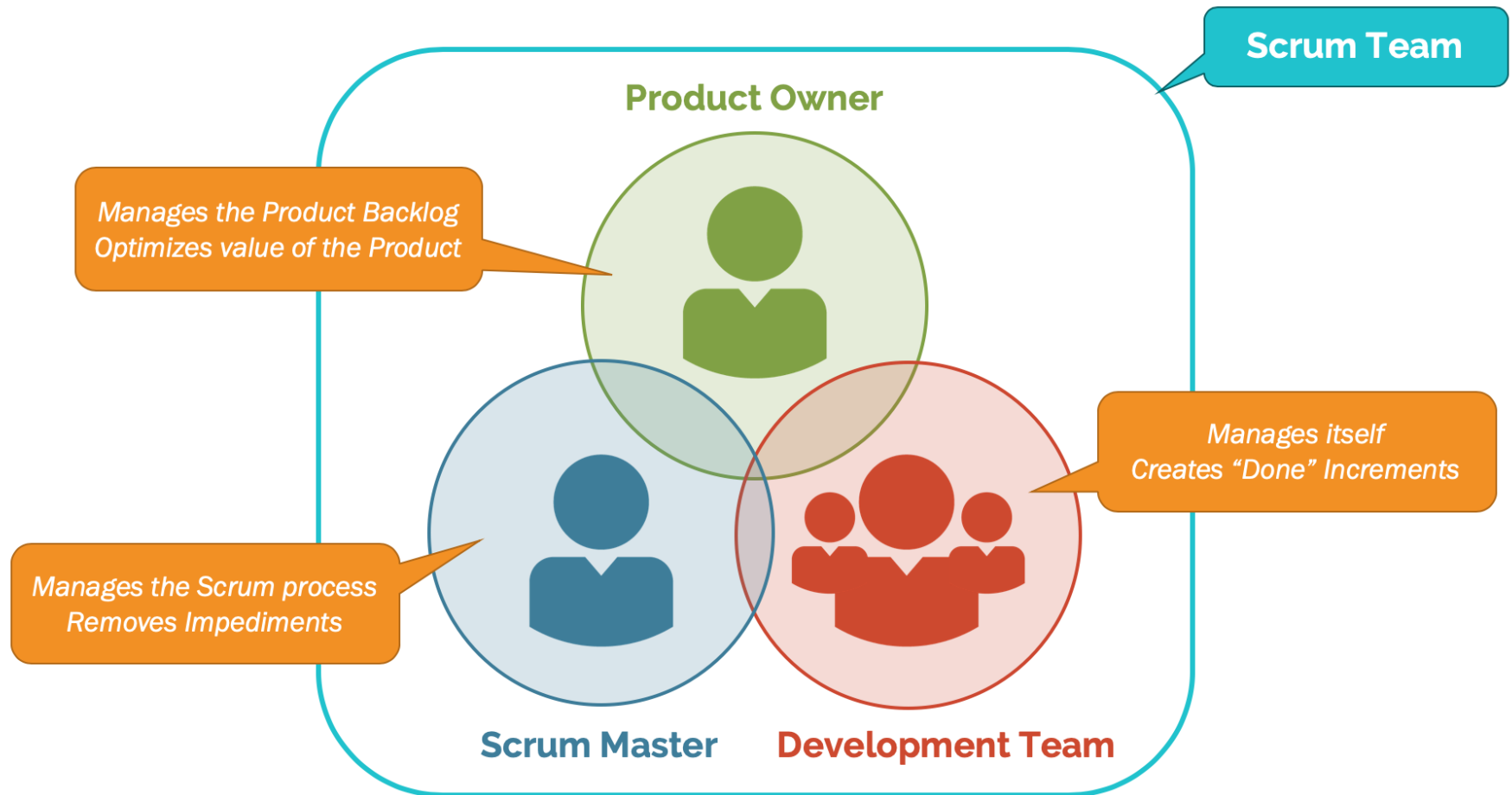
A lightweight framework that provides structure through roles, events, and artifacts.

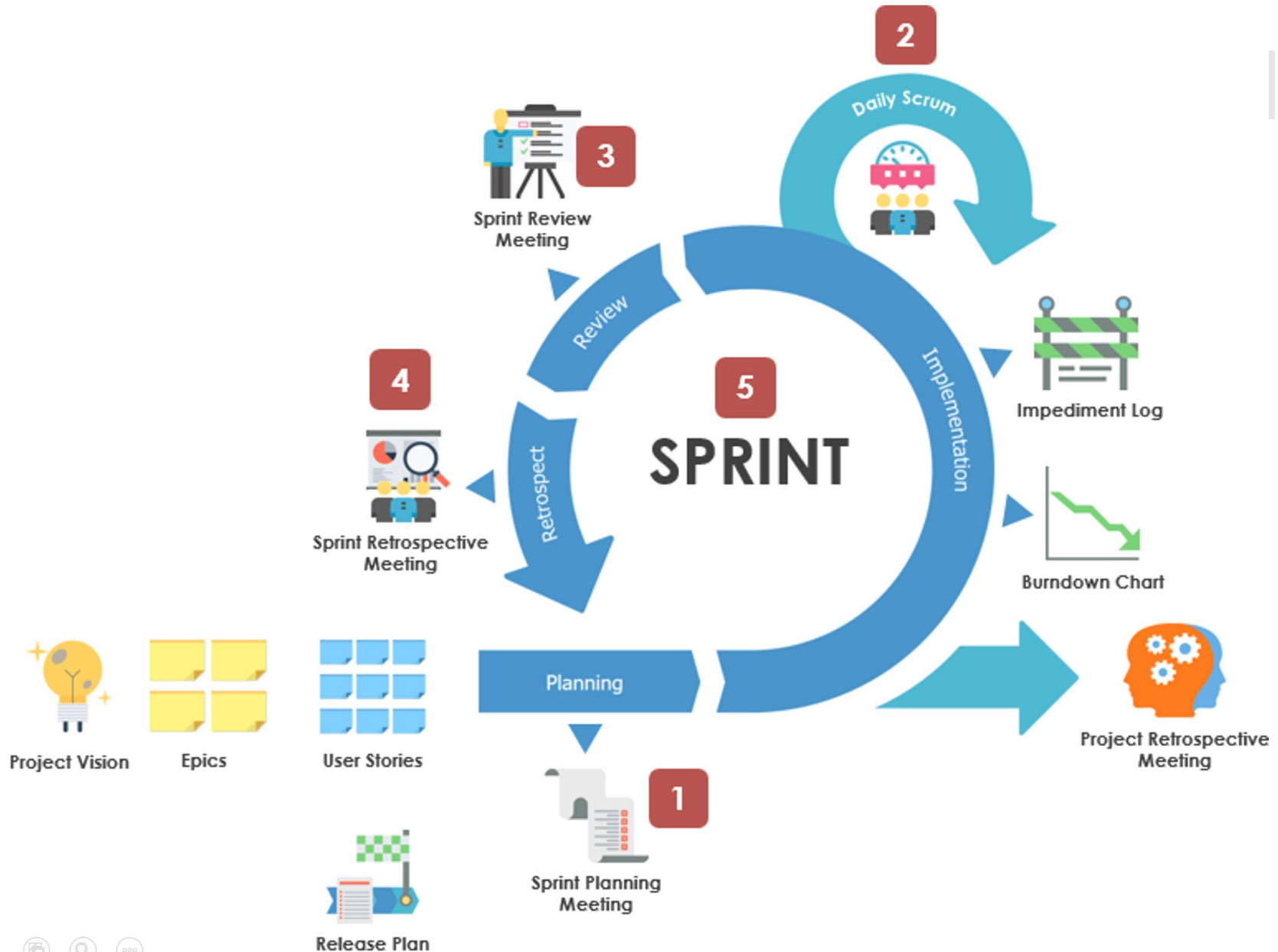
Term	Description
Scrum	The most widely used Agile framework that organizes work into short iterations called sprints.
Sprint	A fixed time period (usually 1–4 weeks) during which a team completes a set of tasks.
Product Backlog	A prioritized list of features, enhancements, bug fixes, and requirements for the product.
Sprint Backlog	A subset of Product Backlog items selected for completion during a sprint.
User Story	A simple description of a feature from the end user's perspective.
Development Team	Cross-functional members who design, develop, test, and deliver the product.
Daily Scrum	A short daily meeting (typically 15 minutes) to discuss progress and blockers.

Scrum Framework

Term	Description
Sprint Planning	A meeting where the team selects and plans work for the upcoming sprint.
Sprint Review	A meeting to demonstrate completed work and gather stakeholder feedback.
Sprint Retrospective	A meeting to reflect on the sprint and identify improvements.
Increment	A working product version delivered at the end of a sprint.
Velocity	The amount of work a team completes during a sprint.
Burndown Chart	A chart showing remaining work versus time during a sprint.
Kanban Board	A visual board used to track task status such as To Do, In Progress, and Done.
Epic	A large body of work that can be divided into multiple User Stories.
Continuous Integration (CI)	Frequently merging code changes into a shared repository with automated testing.
Definition of Done (DoD)	A shared agreement on the criteria required for work to be considered complete.

Roles





Common Practice

- ✓ Pair Programming
- ✓ Test-Driven Development (TDD)
- ✓ Continuous Integration (CI)
- ✓ Refactoring
- ✓ Code Review

Benefits

- ✓ Faster delivery
- ✓ Improved customer satisfaction
- ✓ Better teamwork
- ✓ Higher software quality

Challenges

- ✓ Frequent requirement changes
- ✓ Team coordination issues
- ✓ Requires active customer involvement

Step 1. Create an Atlassian Account

- ✓ Visit Atlassian Cloud.
- ✓ Sign up with your email address.
- ✓ Verify your email account.
- ✓ Create your Atlassian site.

Example

Site Name: travelteam
URL: <https://travelteam.atlassian.net>

Step 2. Access Jira

✓ After logging in, Select Jira Software.

The screenshot displays the Jira Software interface. On the left is a navigation sidebar with a red box highlighting the 'Jira' app icon. Below it are sections for 'For you', 'Recent' (showing a 'tuk' space), 'Starred', 'Apps', 'Plans', and 'Spaces' (with a '+ ...' button). The main workspace on the right features a search bar, a '+ Create' button, and a 'Spaces' section with the 'tuk' space selected. Below this are tabs for 'Summary', 'Backlog' (which is active), 'Board', 'Code', 'Timeline', and a list icon. Further down, there's another search bar, a user profile icon, a 'Filter' button, and a section for 'SCRUM 1 스프린트' with 'Add dates' and '(0 work items)' options. At the bottom, a purple triangle icon and the text 'Plan your sprint' are visible.

Step 3. Create a Space

✓ Purpose:

- Organize projects
- Manage teams
- Group related work



For you

Recent

Starred

Apps

Plans

Spaces

Recent



More spaces



Search



+ Create

Spaces



tuk



...



Summary



Backlog



Board



Code



Timeline



Search ba...



Filter



SCRUM 1 스프린트



Add dates

(0 work items)

Plan your sprint

Step 4. Create a Project

The screenshot displays the Jira web interface. On the left is a sidebar with navigation options: 'For you', 'Recent', 'Starred', 'Apps', 'Plans', and 'Spaces'. Under 'Recent', the 'tuk' space is listed. The main content area at the top features a search bar, a '+ Create' button (highlighted with a red box), and a 'Premium trial 29d' badge. Below this is a 'Spaces' section with the 'tuk' space selected. A navigation bar shows various views: 'Summary', 'Backlog' (selected), 'Board', 'Code', 'Timeline', 'Docs', and 'Forms'. Below the navigation bar is another search bar and a 'Filter' button. The main workspace shows a 'SCRUM 1 스프린트' (SCRUM 1 Sprint) section with 'Add dates' and '(0 work items)'. A 'Plan your sprint' instruction is visible, advising to drag work items from the 'Backlog' or create new ones, and to select 'Start sprint' when ready.

Jira

Search

+ Create

Premium trial 29d

Spaces

tuk

Summary Backlog Board Code Timeline Docs Forms

Search backlog

Filter

SCRUM 1 스프린트 Add dates (0 work items)

Plan your sprint

Drag work items from the **Backlog** section or create new ones to work for this sprint. Select **Start sprint** when you're ready.

Step 5. Create Backlog Items

Spaces



Summary **Backlog** Board Code Timeline Docs Forms Development +



Filter



Create



0 of 0 work items visible | Estimate: 0 of 0

☐ **Backlog** (3 work items)

0 0 0

☒ SCRUM-1 make travel UI

해야 할 일 ▾

-

=

☒ SCRUM-2 make view map

해야 할 일 ▾

-

=

☒ SCRUM-3 connect DB

해야 할 일 ▾

-

=



Create

Step 5. Create Backlog Items

Spaces



Summary Backlog Board Code Timeline Docs Forms Development +

Search ba... Filter



+ Create



0 of 0 work items visible | Estimate: 0 of 0

Backlog (3 work items)

0 0 0 Create sprint

SCRUM-1 make travel UI



해야 할 일

-

=



SCRUM-2 make view map

해야 할 일

-

=



SCRUM-3 connect DB

해야 할 일

-

=



+ Create

Step 6. Create subtasks & description

Search

+ Create

Premium trial 29d

Ask Rovo

6

3+

Spaces

tuk

Summary

Backlog

Board

Code

More 4

Search backlog

Filter

+ Create

0 of 0 work items visible | Estimate: 0 of 0

Backlog (3 work items)

000

SCRUM-1 make travel UI

해야 할 일

-

SCRUM-2 make view map

해야 할 일

-

SCRUM-3 connect DB

해야 할 일

-

+ Create

Jira work item

해야 할 일 Agents

Improve Task

Description

Add a description...

Subtasks

0% Done

Work	P...	A...	Status
SCRUM-4 mak e...	=		해야 할 일
SCRUM-5 mak e...	=		해야 할 일

Linked work items

Step 7. Create Sprint

Spaces



Summary Backlog Board Code Timeline Docs Forms Development +

Search ba...



Filter



+ Create



0 of 0 work items visible | Estimate: 0 of 0

- Backlog (3 work items)

0 0 0 Create sprint

☒ ☒ SCRUM-1 make travel UI

+ Epic

해야 할 일

-

=

☐ ☒ SCRUM-2 make view map

해야 할 일

-

=

☐ ☒ SCRUM-3 connect DB

해야 할 일

-

=



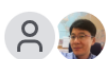
+ Create

Step 8. Start Sprint

Spaces



Summary Backlog Board Code Timeline Docs Forms Development +



Filter

☒ SCRUM 1 스프린트 17 Jun – 19 Jun (1 work item)

0 0 0 Start sprint ...

☒ ☒ SCRUM-1 make travel UI 

+ Epic

진행 중 ▾

- = 👤 ...

+ Create



1 of 1 work item visible | Estimate: 0 of 0

☐ SCRUM 1 스프린트 2 Add dates (0 work items)

0 0 0 Start sprint ...

Plan a sprint by dragging work items into it, or by dragging the sprint footer.

Step 9. Confirm Sprint at Board

Spaces



tuk



...



Summary



Backlog



Board



Code



Timeline



Docs



Forms



Development



Q Search bo...



Filter

Complete sprint



Group v



해야 할 일

+ Create

진행 중 1



검토 중

완료 v

make travel UI

☒ SCRUM-1

Step 10. Assign work to team member

The screenshot shows a web interface for managing work items. At the top, there's a header with a user profile icon, a menu icon, and a search bar. Below the header, there's a navigation bar with tabs: Summary, Backlog (selected), Board, Code, Timeline, Docs, Forms, and Development. A search bar is also present below the navigation bar. The main content area is divided into two sections: a sprint and a backlog. The sprint section is titled "SCRUM 1 스프린트 2" and has a "Start sprint" button. The backlog section is titled "Backlog (2 work items)" and contains two items: "SCRUM-2 make view map" and "SCRUM-3 connect DB". A dropdown menu is open for the "SCRUM-3 connect DB" item, showing a list of team members to assign the work to. The dropdown menu is highlighted with a red circle. The team members listed are: "Unassigned", "Automatic", "Youngkon Lee (Assign to me)", and "youngkon yklee777@tukorea.ac.kr".

tuk

Summary Backlog Board Code Timeline Docs Forms Development

Search ba... Filter

SCRUM 1 스프린트 2 Add dates (0 work items) 0 0 0 Start sprint

Plan a sprint by dragging work items into it, or by dragging the sprint footer.

+ Create

Backlog (2 work items)

SCRUM-2 make view map + Epic

SCRUM-3 connect DB

+ Create

Unassigned

Automatic

Youngkon Lee (Assign to me)

youngkon yklee777@tukorea.ac.kr

Step 11. Team member check the assigned work

[+ Create](#)[Premium trial 29d](#)[Ask Rovo](#)

6



Spaces



tuk

[Summary](#)[Backlog](#)[Board](#)[Code](#)[Timeline](#)[Docs](#)[Forms](#)[Development](#)[Filter](#)[Complete sprint](#)[Group v](#)

해야 할 일 1

make view map

☒ SCRUM-2[+ Create](#)

진행 중 1

make travel UI

☒ SCRUM-1

검토 중

Jira ↔ Github Integration Concept

Link Issues and Code for Seamless Collaboration



✓ SCRUM-17

Password Check

Type: Story
 Status: In Progress
 Assignee: youngkon

Description
 Implement password validation function.

Development

Branches 1
 Commits 3
 Pull requests 1

[View on GitHub](#)

In Jira, you can see

- ✓ Branches
- ✓ Commits
- ✓ Pull Requests
- ✓ Deployment (optional)

Integration (Jira ↔ GitHub Connection)

① Create Issue

② Assign

⑥ Update Status (Done)

③ Create Branch

④ Commit & Push (Include Issue Key)

⑤ Create PR (Merge)



Developer
(youngkon / leeyoungkon)

travel-app Private

feature/SCRUM-17-password-check

Commits

- ✓ a1b2c3d SCRUM-17 Implement password check
- ✓ d4e5f6g SCRUM-17 Add validation logic
- ✓ h7i8j9k SCRUM-17 Update unit tests

Pull Requests

- SCRUM-17 Password check feature Merged
#45 opened 2 days ago

In GitHub, you can reference

- ✓ Issue key in branch/commit/PR
- ✓ Automatically link to Jira issue
- ✓ Status synchronization



Using the Issue Key (e.g., **SCRUM-17**) in branch name, commit message, or PR title automatically connects Jira and GitHub.

Step 0. Git + Jira

The screenshot displays the Jira web interface. On the left sidebar, the 'Apps' menu is highlighted with a red box, and a sub-menu is open, also highlighted with a red box, showing options: 'Manage apps', 'App audit logs', and 'View app requests'. The main content area shows the 'travel app' configuration page. The top navigation bar includes a search bar, a '+ Create' button, and a 'Premium trial' badge. The left sidebar lists various Jira features like 'For you', 'Recent', 'Starred', 'Plans', 'Spaces', 'Recent', 'More spaces', 'Filters', 'Dashboards', and 'Confluence'. The main content area has tabs for 'Summary', 'List', 'Board', 'Backlog', 'Development', 'Forms', and 'Tir'. The 'Board' tab is selected, showing a list of items with checkboxes and user avatars. The items are labeled 'TA-1' and 'TA-2', with the latter having the text 'check password' next to it. A '+ Create' button is at the bottom of the list. On the right side of the board, there are two columns labeled 'IN PROGRESS' and 'IN REVIEW', with a 'Complete sprint' button above them.

Step 0. Git + Jira

The screenshot shows the Jira web interface. On the left is a sidebar with navigation options: 'For you', 'Recent', 'Starred', 'Apps' (with an 'Explore more apps' link), 'Plans', 'Spaces', 'Filters', 'Dashboards', 'Confluence', 'Teams', 'Goals', 'Projects', and 'Customize sidebar'. The top header includes a search bar, a '+ Create' button, and a 'Premium trial' badge. The main content area displays the 'Connect GitHub to Atlassian' setup screen. It features the GitHub and Atlassian logos with a double-headed arrow between them. Below the logos, the text 'Connect GitHub to Atlassian' is followed by the instruction 'Select the GitHub product you are connecting:'. Two buttons are presented: 'GitHub Cloud' (highlighted in blue) and 'GitHub Enterprise Server' (in a light gray box). At the bottom, there is a help link '? How do I check my GitHub product?' and a 'Next' button.

Search + Create Premium trial 29d

For you
Recent
Starred
Apps
Explore more apps
Plans
Spaces
Filters
Dashboards
Confluence
Teams
Goals
Projects
Customize sidebar

Connect GitHub to Atlassian

Select the GitHub product you are connecting:

GitHub Cloud

GitHub Enterprise Server

? How do I check my GitHub product?

Next

Step 0. Git + Jira



Enable Rovo Search for GitHub

Enable Rovo Search to find and use development information across all your Atlassian apps instantly for all of your organizations.

If you don't want to enable this now, you can enable this at any time in [Atlassian Administration](#) [↗](#).

☒ Show GitHub data in Rovo

❗ If you turn this off, we won't show GitHub in Rovo Search, Chat, and Agents.
Your data remains connected for other Atlassian features, but will not be discoverable in Rovo.

+ Things you should know about how Rovo Search for Github handles data, access permissions and compatibility

Next

Step 0. Git + Jira



leeyoungkon is now connected!

Your GitHub data will now be available to access in Atlassian products.

Connected organizations

[Add another organization](#) | [View All](#)

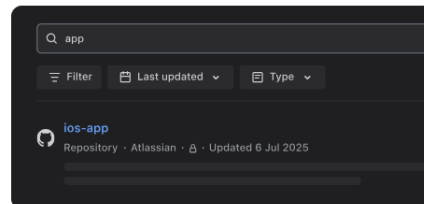


leeyoungkon

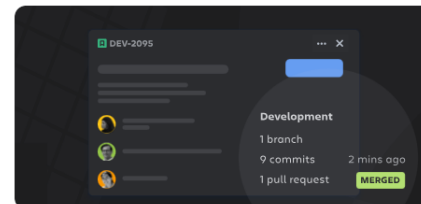
github.com/leeyoungkon

All repos connected

Explore GitHub for Atlassian capabilities

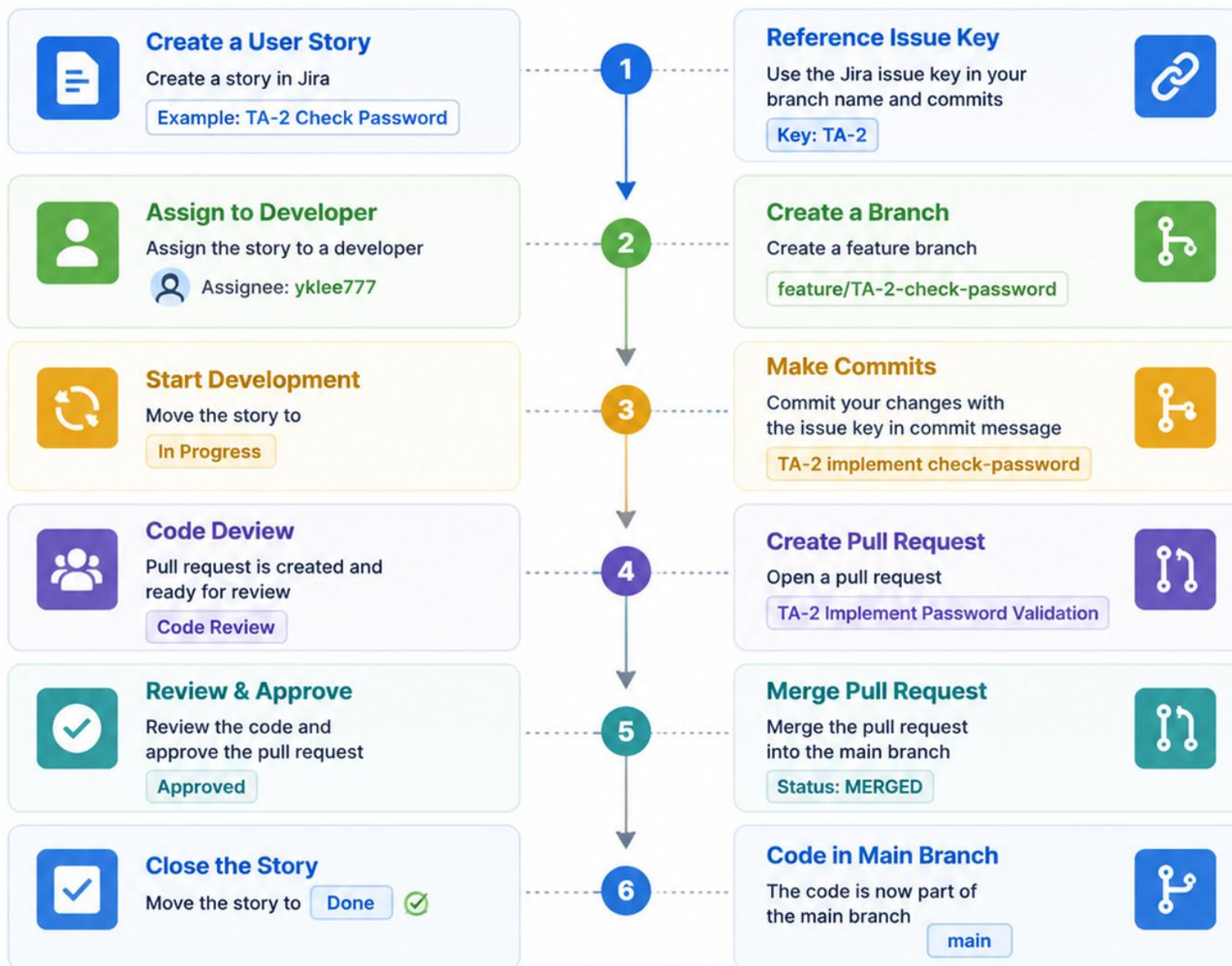


Search for GitHub development information using Rovo Search



View GitHub development information on Jira work items

Practice 2 (Jira + Github)



Step 1. make issue at Jira and assign it to a member

spaces

 **travel app**  ...

 Summary  List  Board  **Backlog**  Development  Forms  Timeline  Docs  Reports +

 Search ba...    Filter   ...

☐ **TA Sprint 1** 17 Jun – 1 Jul (1 work item) 0 0 0 Complete sprint ...

☒ TA-1 search restaurant TO DO  - = 

+ Create

☐ **Backlog** (1 work item)

☐ ☒ TA-2 check password 

+ Create

 **Unassigned**

 Automatic

 Youngkon Lee (Assign to me)

 youngkon
ykle777@tukorea.ac.kr

Step 2. start sprint

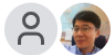
Spaces

 **travel app**  ...



 Summary  List  Board  **Backlog**  Development  Forms  Timeline  Docs  Reports 

 Search ba...



 Filter



 **TA Sprint 2** 1 Jul – 15 Jul (2 work items)

0 0 0

Start sprint ...

☐ ☒ TA-1 search restaurant

TO DO ▾

-

=



☒ ☒ TA-2 check password 

+ Epic

TO DO ▾

-

=



+ Create

Step 3. check sprint by member

Spaces

 **travel app**  ...  

 Summary  List  **Board**  Backlog  Development  Forms  Timeline  Docs  Report

 Search bo...    Filter  Complete sprint  Group  

TO DO 2

search restaurant ...

☒ TA-1  

check password

☒ TA-2  

+ Create

IN PROGRESS

IN REVIEW

+ Create

Step 4. Fork and clone project

- ✓ Fork at leeyoungkon/check-password
 - make your own repository: eg. student1/check-password
- ✓ At your PC, git clone
 - git clone <https://github.com/student1/check-password.git>
- ✓ cd check-password

Step 5. Make branch, commit, push, and PR

- ✓ `git checkout -b feature/TA-2-check-password`
- ✓ `git add .`
- ✓ `git commit -m "TA-2 check password"`
- ✓ `git push origin feature/TA-2-check-password`
- ✓ At github, Compare & Pull Request $\Rightarrow$ generate PR

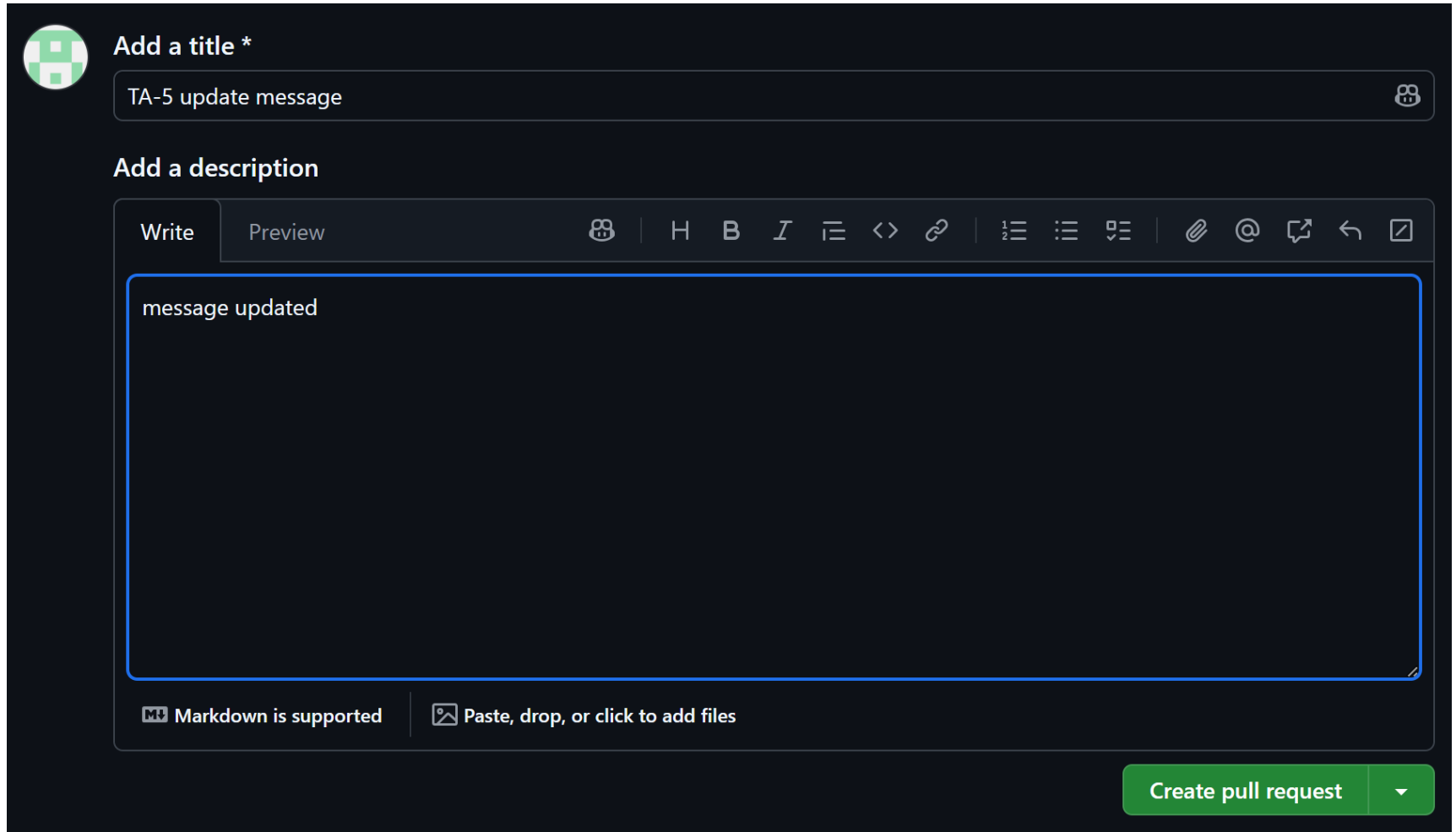
Practice 2 (Jira + Github)

Step 5. Make branch, commit, push, and PR

The screenshot shows the GitHub interface for the repository 'leeyoungkon / check-password'. The repository is public and has 3 branches and 0 tags. A notification bar indicates that the 'feature/TA-5-check-ID' branch had recent pushes 4 seconds ago, with a 'Compare & pull request' button. Below the notification, there is a section for the latest commit, 'Merge pull request #2 from leeyoungkon/feature/TA-3-make-view', which was committed yesterday (e8b46a8) and includes 5 commits. The commit details show a list of files and folders that were added or modified in this commit:

File/Folder	Commit Message	Time
.mvn/wrapper	TA-2-check-password	yesterday
src	TA-3 make view	yesterday
.gitattributes	TA-2-check-password	yesterday
.gitignore	TA-2-check-password	yesterday

Step 5. Make branch, commit, push, and PR



The screenshot shows the GitHub interface for creating a pull request. At the top left is a green circular profile icon. Below it, the text "Add a title *" is followed by a text input field containing "TA-5 update message". To the right of the input field is a GitHub logo icon. Below the title field, the text "Add a description" is followed by a rich text editor. The editor has two tabs: "Write" (active) and "Preview". The "Write" tab shows a toolbar with icons for GitHub logo, heading (H), bold (B), italic (I), list (≡), code (<>), link, table (≡), table (≡), table (≡), link, mention (@), share, undo, and redo. The text area of the editor contains "message updated". At the bottom of the editor, there are two buttons: "Markdown is supported" and "Paste, drop, or click to add files". At the bottom right of the form is a green button labeled "Create pull request" with a dropdown arrow.

 Add a title *

TA-5 update message 

Add a description

Write Preview

 H B I ≡ <>      @   

message updated

 Markdown is supported  Paste, drop, or click to add files

Create pull request 

Step 6. Merge PR

The screenshot displays a GitHub Pull Request (PR) interface. At the top, a navigation bar shows the status: Conversation (0), Commits (2), Checks (0), and Files changed (1). The main content area features a comment from user 'leeyoungkon' stating 'message updated' with a smiley face emoji. Below the comment is a system message: 'Mention @copilot in a comment to make changes to this pull request.' The commit history shows 'leeyoungkon' added 2 commits 6 minutes ago. The commit list includes 'TA-5-check-ID' with hash 'b4fdea2' and 'TA-5-check-ID OK' with hash '17eb1a3'. At the bottom, a green box indicates 'No conflicts with base branch' and 'Merging can be performed automatically.' A green button labeled 'Merge pull request' is visible, along with a link to 'View command line instructions.'

Conversation 0 Commits 2 Checks 0 Files changed 1

leeyoungkon commented [now](#) Owner ...

message updated

😊

🔒 Mention @copilot in a comment to make changes to this pull request.

📁 leeyoungkon added 2 commits [6 minutes ago](#)

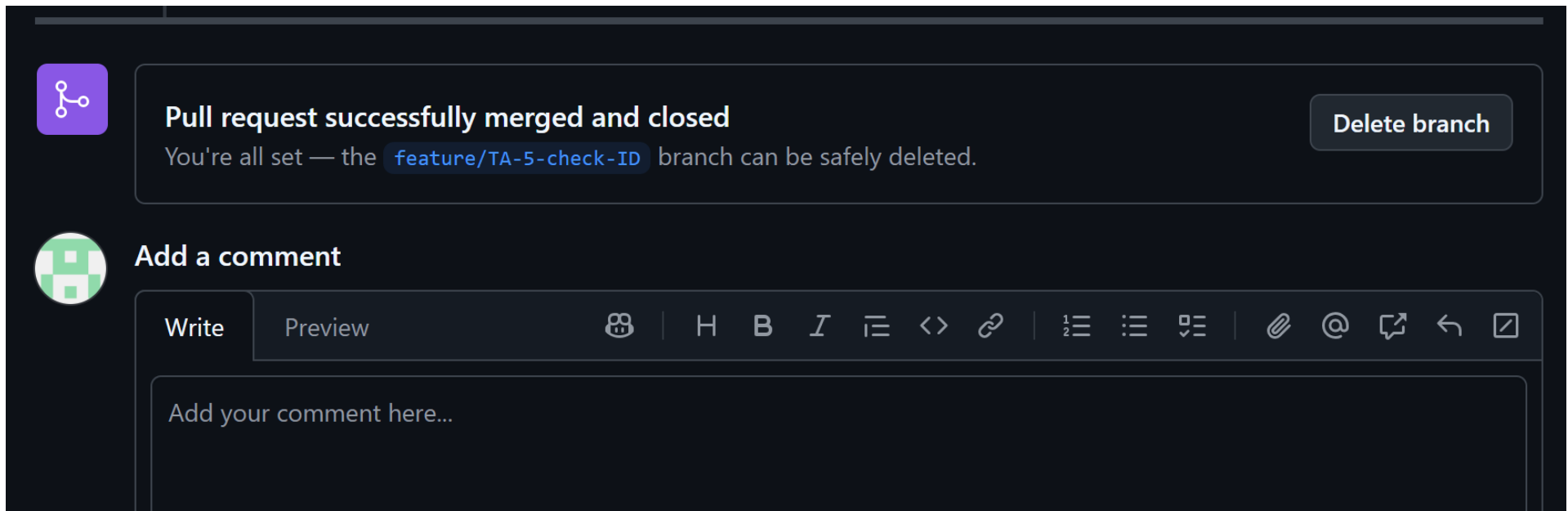
🔗 [TA-5-check-ID](#) [b4fdea2](#)

🔗 [TA-5-check-ID OK](#) [17eb1a3](#)

🔗 **✓ No conflicts with base branch**
Merging can be performed automatically.

[Merge pull request](#) You can also merge this with the command line. [View command line instructions.](#)

Step 7. Confirm result of merge



The screenshot shows a GitHub interface with a dark theme. At the top, a purple icon with a white branching diagram is next to a notification box. The notification box has a title "Pull request successfully merged and closed" and a message "You're all set — the `feature/TA-5-check-ID` branch can be safely deleted." A "Delete branch" button is in the top right of the notification. Below the notification is a section titled "Add a comment" with a green and white checkered profile icon. It has two tabs, "Write" and "Preview", and a rich text editor toolbar with icons for GitHub logo, heading, bold, italic, list, code, link, table, table of contents, quote, attachments, mentions, reactions, undo, and redo. The text area contains the placeholder "Add your comment here..."

 **Pull request successfully merged and closed** [Delete branch](#)

You're all set — the `feature/TA-5-check-ID` branch can be safely deleted.

 **Add a comment**

Write Preview

 | H B I  <>  |    |  @  ↶ 

Add your comment here...

Step 8. Confirm result of merge at Jira

 Add epic /  TA-5

check ID

+

Description

Add a description...

Subtasks

Add subtask

Linked work items

Add linked work item

Agents

 Uses AI. Verify results.



Start work 



Details

Reporter



Youngkon Lee

Development

 [Open with VS Code](#)

 [1 branch](#)

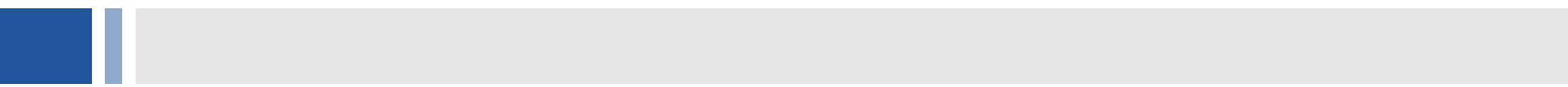
 [3 commits](#) 2 minutes ago

 [Create pull request](#) 

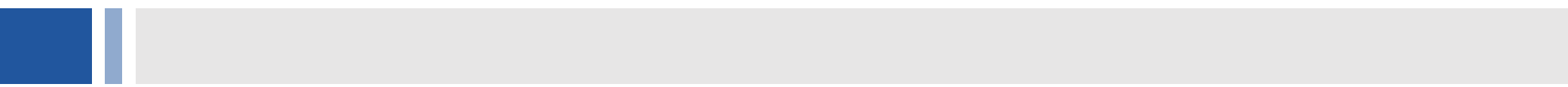
> Automation Rule executions

Created 11 minutes ago

Updated 33 seconds ago



Thank You



Thank You